

**INSTITUTO POLITÉCNICO
DO CÁVADO E DO AVE
ESCOLA SUPERIOR
DE TECNOLOGIA**

Estrutura de Dados Avançados
Relatório do Trabalho Prático

Tiago Ferreira 27980

Data

24-05-2024

Ano

2023 - 2024

Curso

Engenharia de Sistemas Informáticos

Conteúdo

1	Introdução	4
1.1	Enquadramento	4
1.2	Problemas a resolver	4
2	Trabalho Desenvolvido	5
2.1	Análise e Modelação	5
2.2	Representação do Grafo	5
2.3	Tipo de Grafo	6
2.4	Dados das Adjacências	6
3	Implementação	7
3.1	Manipulação de Vértices e Adjacências	7
3.1.1	Inserção de Vértice	8
3.1.2	Inserção de Vértice no Grafo	9
3.1.3	Eliminação de Adjacente	9
3.1.4	Eliminação de Adjacência no Grafo	10
3.2	Criação e Visualização de um Grafo	11
3.2.1	Criação do Grafo	11
3.2.2	Visualização do Grafo	11
4	Análise de Resultados/Reflexão	12
4.1	Operações Não Implementadas	12
4.1.1	Gerir Ficheiro	12
4.1.2	Soma Maxima do Grafo	13
4.2	Considerações sobre Decisões Tomadas	15
5	Conclusões e Trabalho Futuro	16
5.1	Resumo do Trabalho e Resultados Conseguídos	16
5.2	Trabalho a Desenvolver no Futuro	16
6	Anexos	17
7	Refêrencias	18

Lista de Figuras

1	Grafo direcionado e ponderado.	6
---	--	---

Lista de Códigos

1	Função para Inserir Vertice na lista	8
2	Função para Inserir Vertice no Grafo	9
3	Função para eliminar adjacencia da lista.	10
4	Função para eliminar adjacencia do grafo.	10
5	Função para criar grafo.	11
6	Função para mostrar grafo.	11
7	Função para Carregar Grafo a partir de um ficheiro.	12
8	Função para calcular o caminho com a maior soma de vertices.	13

1 Introdução

1.1 Enquadramento

Este projeto de avaliação de realização individual da Unidade Curricular (UC) Estruturas de Dados Avançadas (EDA), integrada no 2^o semestre do 1^o ano, visa o reforço e a aplicação dos conhecimentos adquiridos ao longo do semestre. Com este projeto de avaliação pretende-se sedimentar os conhecimentos relativos à definição e manipulação de estruturas de dados dinâmicas na linguagem de programação C.

1.2 Problemas a resolver

Nesta segunda fase do projeto, concentrei-me em resolver problemas mais complexos, empregando conceitos avançados de teoria dos grafos e programação em C. Os desafios abordados incluem:

1. Modelagem de um problema de otimização utilizando grafos dirigidos.
2. Implementação de uma estrutura de dados adequada para representar um grafo.
3. Desenvolvimento de algoritmos eficientes para busca e análise de caminhos no grafo.
4. Criação de uma solução capaz de calcular o somatório máximo possível de inteiros a partir de uma matriz de dimensões arbitrárias, seguindo regras específicas de conexão entre os elementos.

2 Trabalho Desenvolvido

Nesta seção, é abordado o trabalho desenvolvido para abordar os problemas propostos na fase dois do projeto.

2.1 Análise e Modelação

Para representar o grafo utilizado na resolução do problema, optei pela seguinte estrutura de dados:

- **Grafo:** Estrutura que contém a lista de vértices, o número corrente de vértices e o número máximo de vértices.
- **Vertice:** Estrutura que representa um vértice no grafo, contendo um identificador único, um valor associado, uma flag para marcar se foi visitado durante travessias e um ponteiro para a lista de seus vértices adjacentes.
- **Adjacente:** Estrutura que representa uma aresta do grafo, contendo o identificador do vértice adjacente, um valor associado à aresta, o peso(soma dos vertices conectados) e um ponteiro para o próximo adjacente na lista.

A representação escolhida para o grafo permite uma manipulação eficiente dos vértices e arestas, facilitando a implementação de operações como importação de adjacências, manipulação de vértices e adjacências, travessias e preservação do grafo.

2.2 Representação do Grafo

A seguir, temos trechos de código que ilustram a implementação das estruturas de dados e operações principais:

```
1 typedef struct Adjacente {
2     int id;
3     int valor;
4     struct Adjacente* prox;
5 } Adjacente;
6
7 typedef struct Vertice {
8     int id;
9     int peso;
10    int valor;
11    bool visited;
12    Adjacente* proxAdjacent;
13    struct Vertice* proxVertice;
14 } Vertice;
15
16 typedef struct Grafo {
17     Vertice* inicioGrafo;
18     int numeroVertices;
19     int totVertices;
20 } Grafo;
```

Essas estruturas de dados fornecem uma base sólida para a manipulação eficiente do grafo e a implementação das operações necessárias para resolver o problema proposto.

2.3 Tipo de Grafo

O tipo de grafo que utilizei neste trabalho é direcionado e ponderado.

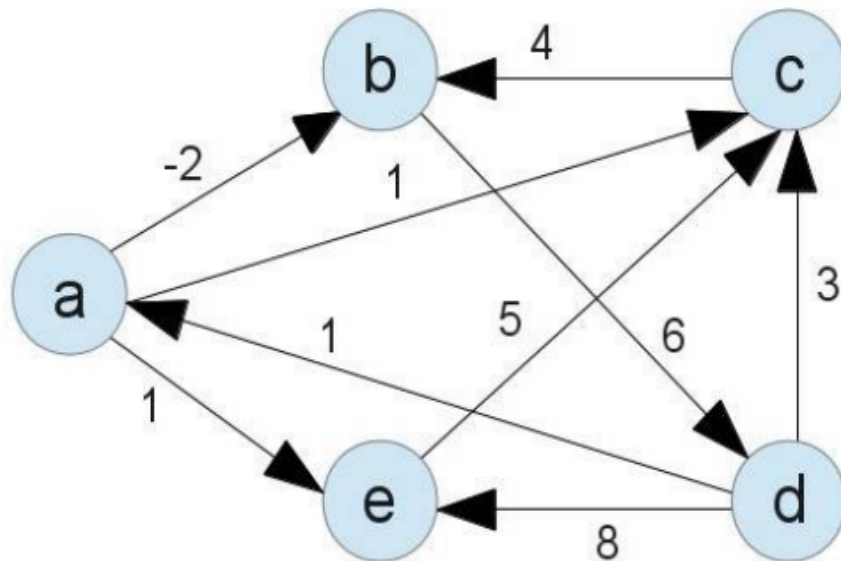


Figura 1: Grafo direcionado e ponderado.

2.4 Dados das Adjacências

Os dados das adjacências são essenciais para representar as relações entre os vértices em um grafo. Nesta implementação, cada adjacência é representada por uma estrutura de dados chamada **Adjacente**. Esta estrutura contém quatro campos:

- **id**: Identificador único do vértice adjacente.
- **valor**: Valor associado à aresta que conecta o vértice atual ao vértice adjacente.
- **peso**: Soma dos vertices conectados (Vertice Origem e Vertice Destino).
- **prox**: Um ponteiro para o próximo adjacente na lista de adjacências do vértice atual.

Essa representação permite armazenar eficientemente as informações sobre as adjacências de cada vértice, facilitando a manipulação e o acesso aos dados ao longo das operações realizadas no grafo.

3 Implementação

Nesta seção, é abordado a implementação do programa desenvolvido para manipulação de grafos. O código-fonte está organizado em seis arquivos distintos, cada um com sua responsabilidade específica:

- **main.c**: Este arquivo contém a função principal (**main**) do programa, responsável por iniciar a execução e chamar outras funções conforme necessário.
- **functions.c**: Aqui estão implementadas as principais funções utilizadas no programa, incluindo aquelas responsáveis por manipular os vértices e as adjacências do grafo.
- **Vertices.c**: Este arquivo contém a implementação das funções relacionadas à manipulação dos vértices do grafo, como inserção, remoção e busca.
- **Adjacencias.c**: Aqui são implementadas as funções responsáveis por lidar com as adjacências dos vértices, incluindo a criação, remoção e consulta de adjacências.
- **Header.h**: Este arquivo contém os protótipos das funções utilizadas em todo o programa, permitindo que os arquivos fonte possam ser compilados separadamente.
- **GereFicheiro.c**: Este arquivo contém uma função relacionada à leitura de grafos em arquivos.

A seguir, há alguns trechos de código exemplificando a implementação de algumas funções do programa.

3.1 Manipulação de Vértices e Adjacências

Nesta subseção, temos a implementação das operações de inserção e eliminação de vértices, bem como a manipulação das adjacências associadas a esses vértices.

A manipulação eficiente de vértices e suas adjacências é essencial para a construção e manutenção de grafos. Ao inserir ou remover vértices, é crucial garantir que as estruturas de dados que representam as adjacências permaneçam consistentes e atualizadas. As funções apresentadas a seguir desempenham um papel fundamental nesse processo, interagindo de maneira coordenada para garantir a integridade do grafo.

As funções **InserirVertice** e **InserirVerticeGrafo** são responsáveis pela inserção de novos vértices no grafo. A primeira função insere um vértice em uma lista encadeada de vértices, mantendo a ordem crescente dos identificadores dos vértices. Já a segunda função insere um vértice em um grafo, verificando se o grafo tem capacidade para adicionar um novo vértice e, em seguida, chamando a função **InserirVertice** para efetuar a inserção.

Por outro lado, as funções **EliminarAdjacente** e **EliminarAdjacenciaGrafo** são responsáveis pela remoção de adjacências entre vértices. A primeira função remove uma adjacência de uma lista encadeada de adjacências, enquanto a segunda função remove uma adjacência entre dois vértices no grafo, localizando os vértices correspondentes e chamando a função **EliminarAdjacente** para executar a remoção.

3.1.1 Inserção de Vértice

A função `InserirVertice` é responsável por inserir um novo vértice em uma lista encadeada de vértices, mantendo a ordem crescente dos identificadores dos vértices.

Inicialmente, verifica-se se o grafo está vazio. Se estiver vazio, o novo vértice é inserido como o primeiro elemento da lista. Caso contrário, percorre-se a lista para encontrar a posição correta para inserir o novo vértice, respeitando a ordem dos identificadores.

Se o identificador do novo vértice já existir na lista, a função retorna sem fazer nenhuma alteração. Caso contrário, o novo vértice é inserido na posição correta e o número total de vértices no grafo é atualizado.

```
1 Vertice* InserirVertice(Vertice* vertices, Vertice* novo, bool* res) {
2     *res = false;
3     Vertice* anterior = NULL;
4
5     // Grafo vazio
6     if (vertices == NULL) {
7         vertices = novo;
8         return vertices;
9     }
10
11    // Verifica se o vertice ja existe na lista
12    if (ExisteVertice(vertices, novo->id) == 1) {
13        printf("Este vertice ja existe.");
14        return vertices;
15    } else {
16        Vertice* aux = vertices;
17        // Encontra a posicao correta para inserir o novo vertice
18        while (aux != NULL && aux->id < novo->id) {
19            anterior = aux;
20            aux = aux->proxVertice;
21        }
22        *res = true;
23        // Insere o novo vertice na posicao correta
24        if (anterior == NULL) {
25            novo->proxVertice = vertices;
26            vertices = novo;
27        } else {
28            novo->proxVertice = aux;
29            anterior->proxVertice = novo;
30        }
31    }
32    return vertices;
33 }
```

Código 1: Função para Inserir Vertice na lista

3.1.2 Inserção de Vértice no Grafo

A função `InserirVerticeGrafo` é responsável por inserir um novo vértice em um grafo.

Inicialmente, verifica-se se o grafo tem capacidade para adicionar um novo vértice. Se não houver capacidade disponível, a função retorna `NULL`. Caso contrário, verifica-se se o vértice já existe no grafo. Se o vértice já existir, a função retorna sem fazer nenhuma alteração.

Em seguida, a função chama a função `InserirVertice`, que efetua a inserção do vértice na lista encadeada de vértices do grafo. Se a inserção for bem-sucedida, o número total de vértices no grafo é atualizado e a função retorna.

```
1 Grafo* InserirVerticeGrafo(Grafo* grafo, Vertice* novo, int* res) {
2     if (grafo->numeroVertices > grafo->totVertices) {
3         return NULL;
4     }
5     *res = 1;
6     // Verifica se o vertice ja existe no grafo
7     if (ExisteVerticeGrafo(grafo, novo->id) == 1) {
8         return grafo;
9     }
10    // Insere o vertice na lista encadeada de vertices do grafo
11    grafo->inicioGrafo = InserirVertice(grafo->inicioGrafo, novo, res);
12    // Atualiza o numero total de vertices no grafo
13    if (*res == true) {
14        grafo->numeroVertices++;
15    }
16    return grafo;
17 }
```

Código 2: Função para Inserir Vertice no Grafo

3.1.3 Eliminação de Adjacente

A função `EliminarAdjacente` é responsável por remover uma adjacência de uma lista encadeada de adjacências.

A função recebe como parâmetros a lista encadeada de adjacências, o identificador do vértice adjacente a ser removido e um ponteiro para uma variável booleana que indica se a operação foi bem-sucedida ou não.

Inicialmente, verifica-se se a lista de adjacências é nula. Se for, a função retorna `NULL`. Caso contrário, percorre-se a lista para encontrar o adjacente com o identificador especificado. Quando o adjacente é encontrado, ele é removido da lista e a memória alocada para ele é liberada.

Por fim, a função atualiza o valor do ponteiro booleano para indicar que a operação foi bem-sucedida e retorna a lista atualizada de adjacências.

```
1 Adjacente* EliminaAdjacente(Adjacente* lista, int id, bool* res) {
2     *res = false;
3     Adjacente* anterior = NULL;
4
5     if (lista == NULL) return NULL;
6
7     Adjacente* aux = lista;
8
9     // Procura o adjacente com o identificador especificado
10    while (aux != NULL && aux->id != id) {
11        anterior = aux;
12        aux = aux->prox;
13    }
14
15    if (!aux) return lista;
16
17    // Remove o adjacente da lista e libera a memoria alocada para ele
18    if (anterior == NULL) {
19        lista = aux->prox;
20    } else {
21        anterior->prox = aux->prox;
22    }
23
24    free(aux);
25    *res = true;
26    return lista;
27 }
```

Código 3: Função para eliminar adjacência da lista.

3.1.4 Eliminação de Adjacência no Grafo

A função `EliminarAdjacenciaGrafo` é responsável por remover uma adjacência entre dois vértices no grafo.

A função recebe como parâmetros o grafo, os identificadores dos vértices de origem e destino da adjacência a ser removida, e um ponteiro para uma variável booleana que indica se a operação foi bem-sucedida ou não.

Inicialmente, verifica-se se o grafo é nulo. Se for, a função retorna `NULL`. Em seguida, busca-se os vértices de origem e destino no grafo usando a função `ProcurarVerticeGrafo`. Quando os vértices são encontrados, a função `EliminarAdjacente` é chamada para remover a adjacência entre eles.

Por fim, a função atualiza o valor do ponteiro booleano para indicar que a operação foi bem-sucedida e retorna o grafo atualizado.

```
1 Grafo* EliminarAdjacenciaGrafo(Grafo* grafo, int idOrigem, int idDestino, bool*
2     res) {
3     *res = false;
4     if (grafo == NULL) return NULL;
5
6     Vertice* VerticeOrigem = ProcurarVerticeGrafo(grafo, idOrigem);
7     Vertice* VerticeDestino = ProcurarVerticeGrafo(grafo, idDestino);
8
9     VerticeOrigem->proxAdjacent = EliminaAdjacente(VerticeOrigem->proxAdjacent,
10    idDestino, res);
11    return grafo;
12 }
```

Código 4: Função para eliminar adjacência do grafo.

3.2 Criação e Visualização de um Grafo

Esta subseção mostra funções essenciais para a criação e visualização de um grafo. A função **CriaGrafo** inicializa a estrutura de dados do grafo, enquanto a função **MostrarGrafo** permite visualizar a estrutura e os elementos do grafo. Essas funções são cruciais para garantir que o grafo seja corretamente configurado e facilmente compreendido.

3.2.1 Criação do Grafo

A função **CriaGrafo** é responsável por inicializar um novo grafo.

A função recebe como parâmetro o número total de vértices que o grafo poderá conter. Ela aloca memória para a estrutura do grafo e inicializa seus campos: **inicioGrafo** é definido como **NULL**, **numeroVertices** é definido como 0, e **totVertices** é definido como o total de vértices especificado.

Se a alocação de memória for bem-sucedida, a função retorna um ponteiro para o novo grafo. Caso contrário, retorna **NULL**.

```
1 Grafo* CriaGrafo(int total) {
2     Grafo* aux = (Grafo*)malloc(sizeof(Grafo));
3
4     if (aux != NULL) {
5         aux->inicioGrafo = NULL;
6         aux->numeroVertices = 0;
7         aux->totVertices = total;
8     }
9     return aux;
10 }
```

Código 5: Função para criar grafo.

3.2.2 Visualização do Grafo

A função **MostrarGrafo** é responsável por exibir a estrutura do grafo e seus elementos.

A função percorre a lista de vértices do grafo, imprimindo o identificador e o valor de cada vértice. Em seguida, para cada vértice, percorre a lista de adjacências, imprimindo o identificador, o peso e o valor de cada adjacente. Se a lista de vértices estiver vazia, a função imprime a mensagem "Vazio".

```
1 void MostrarGrafo(Vertex* grafo) {
2     Vertex* aux = grafo;
3     if (!aux) {
4         printf("Vazio");
5         return;
6     }
7     while (aux != NULL) {
8         printf("Vertice %d Valor: %d \n", aux->id, aux->valor);
9         Adjacente* auxAdj = aux->proxAdjacent;
10        while (auxAdj != NULL) {
11            printf("\tAdjacente: %d Peso: %d Valor: %d\n", auxAdj->id, auxAdj->
12            peso, auxAdj->valor);
13            auxAdj = auxAdj->prox;
14        }
15        printf("-----");
16        printf("\n");
17        aux = aux->proxVertex;
18    }
19 }
```

Código 6: Função para mostrar grafo.

4 Análise de Resultados/Reflexão

Nesta seção, é discutido algumas operações que não foram implementadas com sucesso e sobre as decisões tomadas durante o desenvolvimento do projeto.

4.1 Operações Não Implementadas

4.1.1 Gerir Ficheiro

Durante o desenvolvimento do projeto, uma das operações que tentei implementar foi a função de carregar o grafo a partir de um ficheiro. Infelizmente, essa função apresentou erros durante a execução. Abaixo, apresentamos o código da função `carregarGrafoDeArquivo` e uma análise dos problemas encontrados.

```
1 bool carregarGrafoDeArquivo(const char* nomeArquivo, Grafo* grafo) {
2     FILE* arquivo = fopen(nomeArquivo, "r");
3     if (arquivo == NULL) {
4         printf("Erro ao abrir o arquivo %s\n", nomeArquivo);
5         return false;
6     }
7
8     int linhas = 0;
9     char linha[100];
10
11     // Ler o arquivo e atribuir os valores aos vertices do grafo
12     while (fgets(linha, sizeof(linha), arquivo) != NULL) {
13         Vertice* novoVertice = NULL;
14         Vertice* verticeAnterior = NULL;
15
16         char* token = strtok(linha, ";");
17         while (token != NULL) {
18             int id = atoi(token);
19
20             // Criar um novo vertice
21             Vertice* vertice = CriaVertice(id);
22             if (vertice == NULL) {
23                 fclose(arquivo);
24                 return false;
25             }
26
27             // Inserir o novo vertice no grafo
28             if (verticeAnterior == NULL) {
29                 // Primeiro vertice na linha
30                 novoVertice = vertice;
31             } else {
32                 // Conectar o vertice anterior ao novo vertice
33                 verticeAnterior->proxVertice = vertice;
34             }
35
36             // Atualizar o vertice anterior
37             verticeAnterior = vertice;
38
39             // Proximo token
40             token = strtok(NULL, ";");
41         }
42
43         // Atualizar a lista de vertices do grafo
44         if (grafo->inicioGrafo == NULL) {
```

```

45     // Primeira linha do arquivo
46     grafo->inicioGrafo = novoVertice;
47 } else {
48     // Conectar o ultimo vertice da linha ao novo vertice
49     Vertice* ultimoVertice = grafo->inicioGrafo;
50     while (ultimoVertice->proxAdjacent != NULL) {
51         ultimoVertice = ultimoVertice->proxAdjacent;
52     }
53     ultimoVertice->proxAdjacent = novoVertice;
54 }
55
56 // Mover para a proxima linha
57 linhas++;
58 }
59
60 fclose(arquivo);
61 return true;
62 }
63
64 #pragma endregion

```

Código 7: Função para Carregar Grafo a partir de um ficheiro.

Ao executar a função, encontramos um problema que impede o carregamento correto dos dados do arquivo para a estrutura de dados do grafo.

4.1.2 Soma Maxima do Grafo

Outra operação que não funciona corretamente é a de encontrar o caminho com a soma máxima em um grafo. A seguir, está o código desta função:

```

1 int AtualizaResultado(Resultado* resultado, int somaAtual, Vertice*
   caminhoAtual) {
2     if (resultado == NULL) return -1;
3     if (somaAtual > resultado->soma) {
4         resultado->soma = somaAtual;
5         resultado->caminho = caminhoAtual;
6     }
7     return 1;
8 }
9
10 int BuscaCaminhoMaximo(Vertice* vertice, int somaAtual, Resultado* resultado) {
11     if (vertice == NULL) {
12         return -1;
13     }
14
15     vertice->visited = true;
16     somaAtual += vertice->valor;
17
18     AtualizaResultado(resultado, somaAtual, vertice);
19
20     Adjacente* adjacente = vertice->proxAdjacent;
21     while (adjacente != NULL) {
22         Vertice* verticeAdjacente = ProcurarVerticeGrafo(resultado->grafo,
23         adjacente->id);
24         if (verticeAdjacente != NULL && !verticeAdjacente->visited) {
25             BuscaCaminhoMaximo(verticeAdjacente, somaAtual, resultado);

```

```
26     adjacente = adjacente->prox;
27 }
28
29     vertice->visited = false;
30     return 1;
31 }
32
33 Resultado* CaminhoSomaMaxima(Grafo* grafo) {
34     Resultado* resultado = (Resultado*)malloc(sizeof(Resultado));
35     if (resultado == NULL) {
36         return NULL;
37     }
38     resultado->soma = 0;
39     resultado->caminho = NULL;
40     resultado->grafo = grafo;
41
42     Vertice* vertice = grafo->inicioGrafo;
43     while (vertice != NULL) {
44         BuscaCaminhoMaximo(vertice, 0, resultado);
45         vertice = vertice->proxVertice;
46     }
47
48     return resultado;
49 }
50
51 void ImprimirSomaMaximaMatriz(Grafo* grafo) {
52     Resultado* resultado = CaminhoSomaMaxima(grafo);
53     if (resultado == NULL) return;
54
55     Vertice* caminho = resultado->caminho;
56     printf("Soma Maxima: %d\n", resultado->soma);
57     printf("Caminho: ");
58
59     while (caminho != NULL) {
60         printf("%d (%d)", caminho->id, caminho->valor);
61         caminho = caminho->proxVertice;
62     }
63     printf("\n");
64
65     free(resultado);
66 }
```

Código 8: Função para calcular o caminho com a maior soma de vertices.

4.2 Considerações sobre Decisões Tomadas

Durante o desenvolvimento do projeto, foi tomada várias decisões que influenciaram a estrutura e a funcionalidade do código. A seguir, é apontado algumas dessas decisões e suas implicações:

- **Estrutura de Dados do Grafo:** Foi utilizado uma estrutura baseada em listas de adjacência para representar o grafo. Essa escolha facilitou a implementação de operações básicas, como inserção e remoção de vértices e adjacências. No entanto, também tornou mais complexa a manipulação e a visualização dos dados, especialmente em operações mais avançadas.
- **Ficheiros:** Foi dividido as funcionalidades do grafo em diferentes ficheiros, (`main.c`, `functions.c`, `Vertices.c`, `Adjacencias.c`, `Header.h`, `GereFicheiro.c`). Essa abordagem modularizou o código, tornando-o mais organizado e fácil de visualizar.
- **Tratamento de Erros:** Foram implementadas algumas verificações básicas de erros, como validação de ponteiros nulos.

Essas decisões refletem o compromisso em desenvolver um código eficiente e organizado, embora que haja áreas que poderiam ser melhoradas com mais tempo e recursos. As lições aprendidas neste projeto serão valiosas para futuros trabalhos em manipulação de grafos e outras estruturas de dados complexas.

5 Conclusões e Trabalho Futuro

5.1 Resumo do Trabalho e Resultados Conseguidos

Neste projeto, foi desenvolvido um programa para a manipulação de grafos, utilizando a linguagem C. A implementação foi modular, com funções distribuídas entre diferentes arquivos para melhorar a organização e manutenção do código. As principais funcionalidades implementadas incluem a criação e inserção de vértices e adjacências, bem como a visualização do grafo.

Os principais resultados alcançados incluem:

- **Estruturação do Grafo:** Conseguimos criar e manipular grafos utilizando listas de adjacência, o que facilitou operações básicas e a representação eficiente do grafo.
- **Funções de Inserção e Eliminação:** Implementamos e testamos com sucesso as funções para inserção de vértices e adjacências, bem como suas respectivas funções de eliminação.
- **Função de Somar o Caminho:** Implementamos e testamos com sucesso as funções para calcular a soma dos vértices num determinado caminho.
- **Visualização do Grafo:** Desenvolvemos uma função para exibir o grafo de maneira estruturada, permitindo a visualização clara dos vértices e suas adjacências.

Embora tenha enfrentado alguns desafios, como a implementação da função de carregamento do grafo a partir de um arquivo e a soma máxima, o trabalho realizado até agora fornece uma base sólida para a manipulação de grafos e pode ser expandido e melhorado no futuro.

5.2 Trabalho a Desenvolver no Futuro

Para aprimorar e concluir o projeto, várias áreas podem ser desenvolvidas no futuro:

- **Correção das Funções:** Algumas funções não foram implementadas com sucesso devido a erros na execução. Essas funções precisam ser revistas e corrigidas, garantindo que o programa funcione corretamente.
- **Tratamento de Erros:** O tratamento de erros pode ser aprimorado para incluir mais verificações e mensagens de erro detalhadas. Isso ajudará na detecção e correção de bugs, tornando o código mais robusto e confiável.
- **Documentação e Testes:** A documentação do código pode ser expandida para incluir descrições detalhadas de todas as funções e suas interações. Além disso, a criação de um conjunto abrangente de testes unitários ajudará a garantir que todas as funções funcionem conforme o esperado e facilitará a detecção de regressões.

Em conclusão, o trabalho realizado até agora proporciona uma base sólida para a manipulação de grafos, mas há várias áreas que podem ser desenvolvidas e aprimoradas no futuro. A implementação das melhorias e funcionalidades adicionais sugeridas tornará o programa mais robusto, eficiente e aplicável a uma gama maior de problemas e aplicações.

6 Anexos

PDF do Enunciado do Trabalho:



7 Refêrencias

Graph Traversals - BFS & DFS - Breadth First Search and Depth First Search de Abdul Bari.